

Architecture of Android

Android operating system is a stack of software components which is roughly divided into five sections and four main layers as shown below in the architecture diagram.

APPLICATIONS

Home Dialer SMS/MMS IM Browser Camera Alarm Calculator
Contacts Voice Dial Email Calendar Media Player Photo Album Clock ...

APPLICATION FRAMEWORK

Activity Manager Window Manager Content Providers View System Notification Manager
Package Manager Telephony Manager Resource Manager Location Manager ...

LIBRARIES

Surface Manager Media Framework SQLite WebKit Libc
OpenGL|ES Audio Manager FreeType SSL ...

ANDROID RUNTIME

Core Libraries

Dalvik Virtual Machine

HARDWARE ABSTRACTION LAYER

Graphics Audio Camera Bluetooth GPS Radio (RIL) WiFi ...

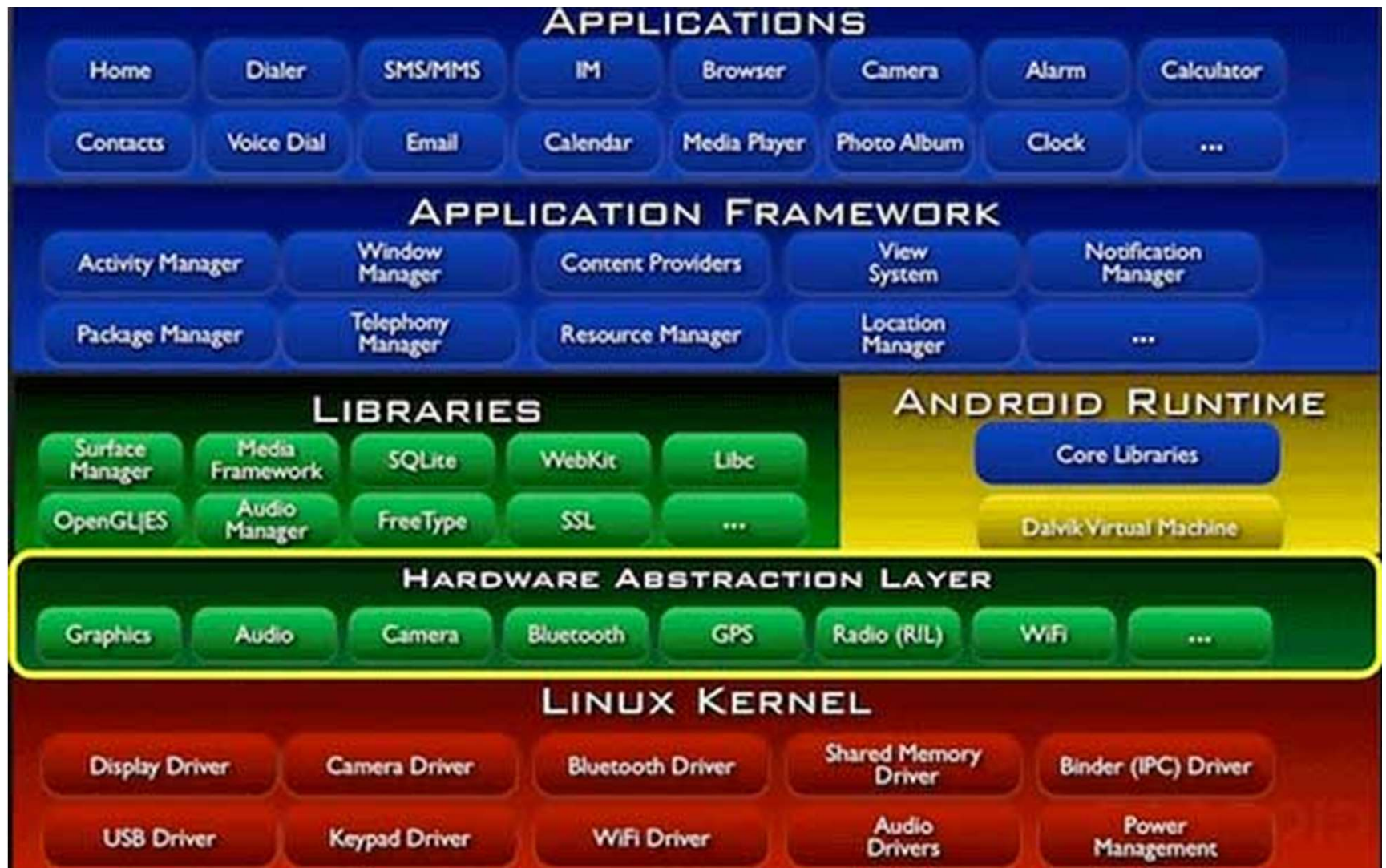
LINUX KERNEL

Display Driver Camera Driver Bluetooth Driver Shared Memory Driver Binder (IPC) Driver
USB Driver Keypad Driver WiFi Driver Audio Drivers Power Management

Java/JNI

C/C++

C



What is Android?

Android provides a layered stack for mobile devices, including...

- Hardware sensors, transceivers, storage, and multi-core processors
- A variant of the Linux OS optimised for power conservation and local IPC
- An optimised Java Virtual Machine i.e. DVM, a subset of Java libraries running on DVM, native C/C++ libraries and a hardware abstraction layer
- Middleware, including
 - GUIs; Telephony services,
 - Camera; Multimedia,
 - App framework; App distribution, etc
- Packaged Apps

1. Linux Kernel

- At the bottom of the layers is Linux. This provides a level of abstraction between the device hardware and it contains all the essential hardware drivers like camera, keypad, display etc.
- Also, the kernel handles all the things that Linux is really good at such as networking and a vast array of device drivers, which take the pain out of interfacing to peripheral hardware.
- The Linux Kernel is a battle-tested piece of software that has been used in developing operating systems for devices of wide range, from supercomputers to small gadgets. It has limited processing abilities like small networked gadgets for the Internet of Things (IoT).

With regards to Android, the Kernel is responsible for many foundational functionalities including but not limited to these:

A. Device Drivers

B. Memory Management

C. Process Management

A. Device Drivers

- The Linux Kernel houses the drivers needed to make it possible for the operating system to work with different hardware components. These drivers provide a standard interface with which hardware components sourced from different manufacturers can work with.
- This makes it possible for device manufacturers to source different components, such as Bluetooth components, Wifi components, camera components. As long as the manufacturers match the Android standard specifications, integration is seamless.

LINUX KERNEL

**Usb
Driver**

1

**Bluetooth
Driver**

2

**Wifi
Driver**

3

**Display
Driver**

4

**Audio
Driver**

5

**Power
Manager**

6

**Flash
Memory**

7

**Binder
IPC Driver**

8

1) USB Driver

Linux provides Android with a means to interface with USB devices. Modern devices come with different USB ports, including USB 2.0, 3.0 and new versions of USB, including USB-C. These drivers make it possible to use the USB port to charge, transfer live data such as logs from the Android devices, and interact with the android file system.

2) Bluetooth Driver

Linux Kernel provides support for interfacing with Bluetooth hardware components. It provides a way to read and write data received from supported bluetooth radio frequencies. It also provides a set of facilities for Android to configure Bluetooth.

3) Wifi Driver

The Linux kernel provides drivers to integrate the WiFi networking hardware components. WiFi components embedded in mobile devices enable Android devices to connect to wifi networks. The driver enables the wifi components to broadcast wifi networks and create hotspots.

4) Display Driver

Android makes it possible to interface with display components. For most devices, the interface component is an LCD touch-screen. It allows support for configuring and drawing pixels.

5) Audio Driver

Android devices commonly come with hardware components for audio input and output. Audio drivers in the kernel enable the Android system to use audio received from these components and also produce audio output.

6) Power Manager

Most Android devices are used while disconnected from power outlets. They thus depend on batteries to power them for a large chunk of their usage. Linux Kernel comes with a power management system that's configurable to meet the needs of the devices using it. Android OS uses the power manager to make other components on the device power-aware. It does this by broadcasting various power-related states. These states are Standby, Sleep, and Low-Battery. On Android, the power manager is tweaked to default to sleep mode to ensure maximum battery life.

- The Power Manager exposes means for applications to react to different power modes. Applications can also change their behavior to match the current power state of the device.
- An application can also request to change the default power policies. Applications can achieve the desired functionality, such as keeping the hardware components active. An example is keeping the screen active when reading a book to ensure a user isn't interrupted. Another example is keeping the audio components turned on when listening to music in the background.

7) Flash Memory

Most Android devices use flash memory as a means of storage. Flash memory is fast and takes less space making it perfect for small devices. Linux kernel provides a means for Android devices to read and write into flash memory. It provides a means to partition the memory in such a way that the OS and other applications can easily and efficiently share the memory resource.

8) Binder

Android hosts many applications and system components that each run in their processes. In most cases, these processes should be isolated from each other to prevent interference and data corruption. Yet, there are instances that we wish to pass data from one process to another. The Linux kernel enables data sharing functionality by providing binder drivers.

B. Memory Management

- Another responsibility of the Linux Kernel is memory management. As different applications run, the Kernel ensures the memory space they use doesn't conflict and overwrite each other.
- It also helps ensure that all running apps get adequate memory to function, making sure no single app takes too much space.

C. Process Management

- Every app in Android runs in a process. The Kernel is also responsible for managing processes. This means it's responsible for creating, pausing, stopping, shutting, or killing down processes.
- The Kernel enables various functionalities such as running multiple processes at the same time, communicating between processes, running processes in the background, and so on.
- As each process requires its own memory space to function correctly, the Kernel ensures that memory spaces allotted for each process are protected from other processes.

- It also ensures that resources like RAM allotted to processes are freed up when the processes are shut down.
- The Linux Kernel is also responsible for distributing work to the processors present in the device. This makes it possible to maximize the performance of devices with multiple cores as different apps will have processes being run on a different core.

2. Libraries

The Platform Libraries includes various C/C++ libraries and Java based libraries such as Media, Graphics, Surface Manager, OpenGL etc. to provide a support for android development.

- **MediaFramework:** Android also natively supports popular media codecs, making it easy for apps created on the Android platform to use/play multimedia components out of the box.
- **Surface manager:** Responsible for managing access to the display subsystem. The surface manager is responsible for ensuring the smooth rendering of application screens. It does this by composing 2D and 3D graphics for rendering. It further enables this by doing off-screen buffering.

- **SQLite:** Android has an SQLite database that enables applications to have very fast native database functionality without the need for third party libraries.
- **OpenGL:** Android also comes with the OpenGL graphics system. It's a C library that helps Android use hardware components in the real-time rendering of 2D and 3D graphics.
- **Web-Kit:** This is an open-source Browser engine used as a basis to build browsers. The default Android browser before version 4.4 KitKat uses it for rendering web pages. It enables application developers to render web components in the view-system by using WebView. This enables apps to integrate web components into their functionality.

- **SSL (Secure Sockets Layer)** is security technology to establish an encrypted link between a web server and a web browser.
- **SGL:** Android comes with a graphics library implemented in low-level code that efficiently renders graphics for the android platform. It works with the higher-level components of the Android framework Android graphics pipeline.
- **Libc:** The core of Android contains libraries written in C and C++, which are low-level languages meant for embedded use that help in maximizing performance. Libc provides a means to expose low-level system functionalities such as Threads, Sockets, IO, and the like to these libraries.

3. Android Runtime

- This section provides a key component called Dalvik Virtual Machine (DVM) which is a kind of Java Virtual Machine specially designed and optimized for Android.
- The DVM makes use of Linux core features like memory management and multi-threading, which is intrinsic in the Java language. The DVM enables every Android application to run in its own process, with its own instance of the DVM.
- Programs for Android are generally written in Java and compiled to bytecode for the Java virtual machine, which is then translated to Dalvik bytecode and stored in .dex (Dalvik EXecutable) and .odex (Optimized Dalvik EXecutable) files. The compact Dalvik Executable format is designed for systems that are constrained in terms of memory and processor speed.

Compilation and Execution

Source

1. Java Source Code (.java)

Java Compiler (javac)

2. Java Byte Code (.class)

Dex Compiler

3. Dalvik Byte Code (.dex)

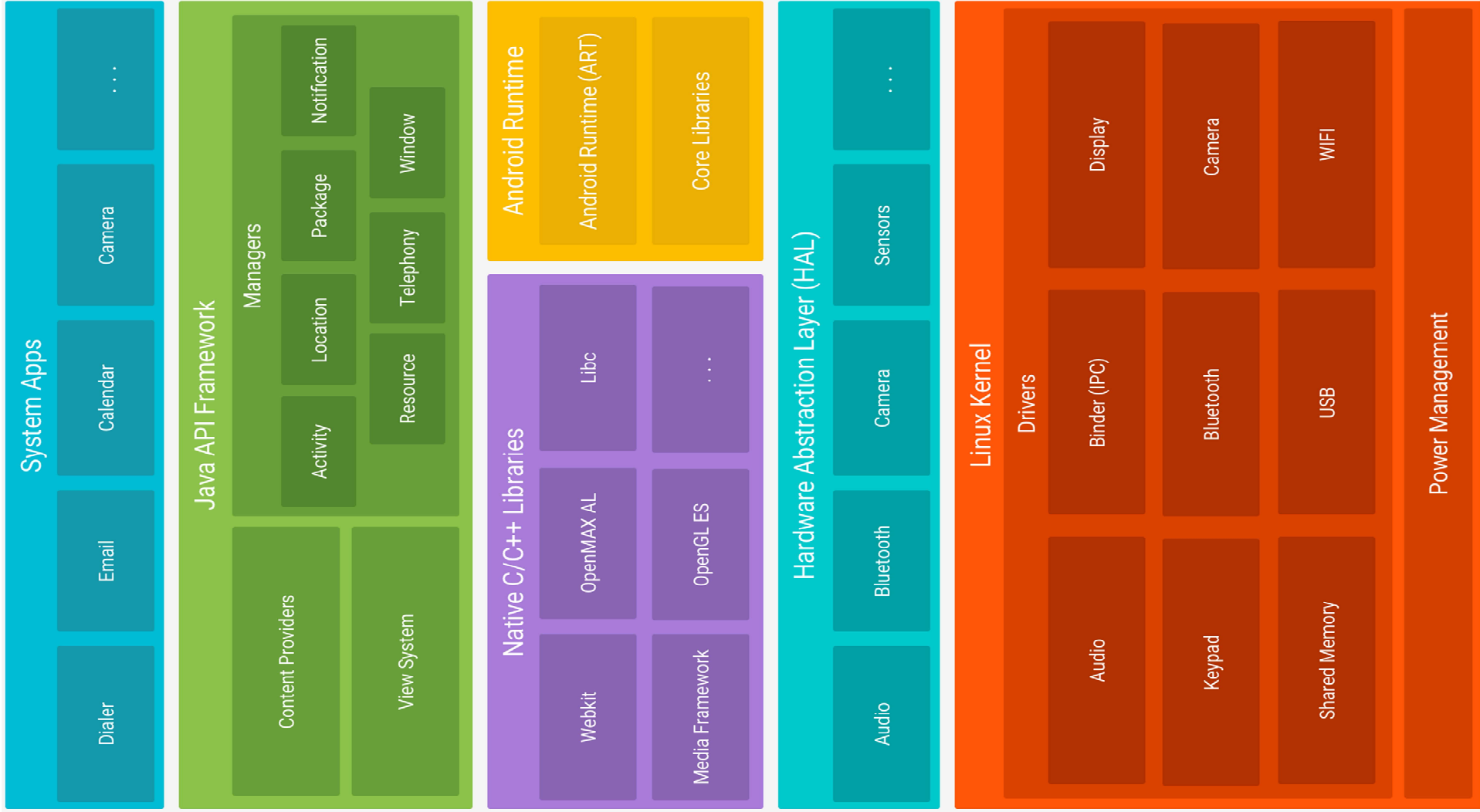
Dalvik Virtual Machine (DVM)

4. Binary Machine Code

- The Android runtime also provides a set of core libraries which enable Android application developers to write Android applications using standard Java programming language.
- The runtime translates code written by programmers into machine code that does computations and utilizes android framework components to deliver functionality. Android hosts multiple applications and system components that each run in their processes.

Android Runtime (ART)

- DVM was an integral part of the Android software stack in the Android versions 4.4 "KitKat" and earlier. But now, it is discontinued and unsupported.
- The successor of Dalvik is ART, which uses the same bytecode and .dex files (but not .odex files).
- It aims at performance improvements and transparency to the end users.
- The new runtime environment was included for the first time in Android 4.4 "KitKat" as a technology preview and replaced Dalvik entirely in later versions.
- Android 5.0 "Lollipop" is the first version in which ART is the only included runtime.
- ART is written in C/C++.



4. Application Framework

- The Application Framework layer provides many higher-level services to applications in the form of Java classes. Application developers are allowed to make use of these services in their applications.
- The Android OS exposes the underlying libraries and features of the Android device that are using a Java API. This is what is known as the Android framework. The framework exposes a safe and uniform means to utilize Android device resources.

APPLICATION FRAMEWORK

**Activity
Manager**

1

**Window
Manager**

2

**Location
Manager**

3

**Telephony
Manager**

4

**Resource
Manager**

5

View System

6

**Notification
Manager**

7

**Package
Manager**

8

**Content
Providers**

9

1) Activity Manager

- Applications use the Android activity component for presenting an entry point to the app.
- Android Activities are the components that house the user interface that app users interact with. As end-users interact with the Android device, they start, stop, and jump back and forth across many applications. Each navigation event triggers activation and deactivation of many activities in respective applications.
- The Android ActivityManager is responsible for predictable and consistent behavior during application transitions.

- The ActivityManager provides a slot for app creators to have their apps react when the Android OS performs global actions. Applications can listen to events such as device rotation, app destruction due to memory shortage, an app being shifted out of focus, and so on.
- Some examples of the way applications can react to these transitions include pausing activity in a game, stopping music playing during a phone call.

2. Window Manager

- Android can determine screen information to determine the requirements needed to create windows for applications. Windows are the slots where we can view our app user interface. Android uses the Window manager to provide this information to the apps and the system as they run so that they can adapt to the mode the device is running on.
- The Window Manager helps in delivering a customized app experience. Apps can fill the complete screen for an immersive experience or share the screen with other apps. Android enables this by allowing multi-windows for each app.

3) Location Manager

- Most Android devices are equipped with GPS devices that can get user location using satellite information to which can go all the way to meters precision. Programmers can prompt for location permission from the users, deliver location, and aware experiences.
- Android is also able to utilize wireless technologies to further enrich location details and increase coverage when devices are enclosed spaces. Android provides these features under the umbrella of the Location-Manager.

4) Telephony Manager

Most Android devices serve a primary role in telephony. Android uses Telephone Manager to combine hardware and software components to deliver telephony features. The hardware components include external parts such as the sim card, and device parts such as the microphone, camera, and speakers. The software components include native components such as dial pad, phone book, ringtone profiles. Using the TelephoneManager, a developer can extend or fine-tune the default calling functionality.

5) Resource Manager

Android app usually come with more than just code. They also have other resources such as icons, audio and video files, animations, text files, and the like. Android helps in making sure that there is efficient, responsive access to these resources. It also ensures that the right resources are delivered to the end-users. For example, the proper language text files are used when populating fields in the apps.

6) View System

Android also provides a means to easily create common visual components needed for app interaction. These components include widgets like buttons, image holders such as `ImageView`, components to display a list of items such as `ListView`, and many more. The components are premade but are also customizable to fit app developer needs and branding.

7) Notification Manager

The Notification Manager is responsible for informing Android users of application events. It does this by giving users visual, audio or vibration signals or a combination of them when an event occurs. These events have external and internal triggers. Some examples of internal triggers are low-battery status events that trigger a notification to show low battery. Another example is user-specified events like an alarm. Some examples of external triggers include new messages or new wifi networks detected.

8) Package Manager

- Android also provides access to information about installed applications. Android keeps track of application information such as installation and uninstallation events, permissions the app requests, and resource utilization such as memory consumption.
- This information can enable developers to make their applications to activate or deactivate functionality depending on new features presented by companion apps.

9) Content Provider

Android has a standardized way to share data between applications on the device using the content provider. Developers can use the content provider to expose data to other applications. For example, they can make the app data searchable from external search applications. Android itself exposes data such as calendar data, contact data, and the like using the same system.

5. Applications Layer

You will find all the Android application at the top layer. You will write your application to be installed on this layer only. Examples of such applications are Contacts Books, Browser, Games etc.

Android, by default, comes with a set of applications that make android devices usable from the offset.

1. **Home:** The Homepage on Android consists of launcher icons for commonly used applications that the end-user may want quick access to. You can start the apps by clicking on the launchers of these apps. At the very top of the screen, you have widgets that show network, battery level, date, and time.
2. **Contacts:** Android, by default, provides a means to store and retrieve contacts. Contact information is shared across other apps to enhance functionality.

3. Messages: Android provides the capability to send and receive SMS messages.

4. Email: Android comes with native support for email services. Setting up an Android device requires a Gmail account. Setting up Gmail activates other email-dependent components on Android devices. Some email dependent features include security and recovery mechanisms. Another email dependent feature is access to the Play Store, a marketplace for Android applications.

5. Browser: Android comes with a default browser.

Thank You